임베딩 형태 비교: Dense vs Sparse

개요

RAG에서 텍스트는 **임베딩 벡터**로 변환되어 저장/검색됩니다. 임베딩은 크게 **Dense**와 **Sparse** 두 가지 형태가 있습니다.

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          임베딩의 두 가지 형태                            │
│                                                                          │
│  텍스트: "김철수는 파이썬으로 프로젝트를 개발합니다"                       │
│                                │                                         │
│                ┌───────────────┴───────────────┐                         │
│                ▼                               ▼                         │
│        [Dense Encoder]                  [Sparse Encoder]                 │
│        (BERT, OpenAI)                   (BM25, TF-IDF)                   │
│                │                               │                         │
│                ▼                               ▼                         │
│   ┌──────────────────────┐         ┌──────────────────────────┐         │
│   │ [0.023, -0.089,      │         │ indices: [0, 2, 4, 6, 8] │         │
│   │  0.041, -0.015,      │         │ values:  [2.5, 3.1, 3.8, │         │
│   │  0.078, -0.023,      │         │           1.2, 2.9]      │         │
│   │  ...]                │         │                          │         │
│   │                      │         │ 의미:                    │         │
│   │ 1536개 값 전부 채움  │         │ 김철수:2.5, 파이썬:3.1   │         │
│   └──────────────────────┘         │ 쿠버네티스:3.8 ...       │         │
│                                    └──────────────────────────┘         │
│        Dense Vector                     Sparse Vector                    │
│        (밀집 벡터)                      (희소 벡터)                      │
└──────────────────────────────────────────────────────────────────────────┘
```

Part 1: Dense Embedding (밀집 벡터)

실제 형태

```
# OpenAI text-embedding-3-small 출력 예시 (1536차원)
dense_embedding = [
    0.0234, -0.0891, 0.0412, -0.0156, 0.0789, -0.0234, 0.0567, -0.0123,
    0.0345, -0.0678, 0.0123, -0.0456, 0.0891, -0.0234, 0.0567, -0.0789,
    # ... 1536개 값 전부 채워져 있음
]
```

특징

```
┌──────────────────────────────────────────────────────────────────────┐
│                        Dense Embedding 특징                          │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  [시각화 - 모든 차원에 값이 존재]                                     │
│  ▓▓▓
```

특징

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          Sparse Embedding 특징                           │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  [시각화 - 대부분 0, 일부만 값 존재]                                      │
│  ▓·▉·▉·▒·▓·····                                                          │
│  (30,000개 중 6개만 값이 있음, 나머지는 0)                                │
│                                                                          │
│  차원 수:        30,000 ~ 100,000+ (어휘 크기)                            │
│  값 범위:        0 ~ ∞ (TF-IDF, BM25 점수)                                │
│  비영값 비율:    0.01% ~ 1% (대부분 0)                                    │
│  저장 크기:      (index, value) pairs만 저장                              │
│                   예: 100쌍 × 8 bytes = 800 bytes ≈ 0.8KB/문서            │
│                                                                          │
│  생성 방법:      통계 기반 (TF-IDF, BM25) 또는 학습 (SPLADE)              │
│  의미:           각 차원 = 특정 단어 (해석 가능!)                         │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

BM25 계산 예시

```
문서: "김철수는 파이썬으로 프로젝트를 개발합니다"

[BM25 Sparse Vector 계산 결과]
{
    '김철수': 2.5,       # IDF 높음 (드문 단어)
    '파이썬으': 3.1,     # IDF 높음
    '프로젝트': 2.9,     # IDF 중간
    '개발합니다': 1.8,   # IDF 낮음 (흔한 단어)
    '는': 0.3,           # IDF 매우 낮음
    '를': 0.2            # IDF 매우 낮음
}

# 어휘 30,000개 중 6개만 값이 있음 = 99.98%가 0
```

장단점

장점	단점
정확한 키워드 매칭	동의어 처리 못함
"iPhone 15 Pro" 정확 매칭	의미적 유사성 무시
희귀 단어 잘 찾음	"강아지" ≠ "puppy"

장점	단점
해석 가능	
저장 공간 작음	

Part 3: Dense vs Sparse 직접 비교

```
┌──────────────────────────────────────────────────────────────────────┐
│                    Dense vs Sparse 시각적 비교                        │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  [Dense Embedding - 8차원 예시]                                       │
│                                                                      │
│  차원:   0      1      2      3      4      5      6      7           │
│  값:   0.023 -0.089  0.041 -0.015  0.078 -0.023  0.056 -0.012        │
│  시각:  ▓▓▓
```

```
│                                                                    │
│  검색: 코사인 유사도로 가장 가까운 벡터 찾기                          │
│        similarity = cos(query_vec, doc_vec)                         │
│                                                                    │
│  결과: "기쁜 때의 추억" (의미가 비슷하면 매칭!)                        │
│        "즐거운 시간" (동의어도 매칭!)                                  │
│                                                                    │
│  인덱스: ANN (Approximate Nearest Neighbor)                          │
│          - HNSW, IVF, ScaNN 등                                       │
│                                                                    │
├────────────────────────────────────────────────────────────────────┤
│                                                                    │
│  [Sparse Vector Search]                                              │
│                                                                    │
│  Query: "iPhone 15 Pro 가격"                                         │
│         → Sparse Vector: {iPhone: 3.2, 15: 1.5, Pro: 2.8, 가격: 2.1}  │
│                                                                    │
│  검색: Inverted Index로 해당 단어 포함 문서 찾기                       │
│        score = Σ (query_weight × doc_weight)                         │
│                                                                    │
│  결과: "iPhone 15 Pro 출시 가격 정보" (정확한 키워드 매칭)             │
│        ✗ "애플 최신폰 비용" (다른 단어라 매칭 안됨)                    │
│                                                                    │
│  인덱스: Inverted Index (Elasticsearch, Lucene)                      │
│          단어 → 문서 목록 매핑                                         │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

Part 5: Hybrid Search

```
┌────────────────────────────────────────────────────────────────────┐
│                        Hybrid Search 구조                            │
│                                                                    │
│                   Query: "김철수 파이썬 프로젝트"                      │
│                                  │                                  │
│                     ┌────────────┴────────────┐                     │
│                     ▼                         ▼                     │
│              [Dense Encoder]              [Sparse Encoder]           │
│              (BERT, OpenAI)               (BM25, SPLADE)             │
│                     │                         │                     │
│                     ▼                         ▼                     │
│   Dense: [0.02, -0.08, 0.04, ...]    Sparse: {김철수:2.5, 파이썬:3.1}  │
│                     │                         │                     │
│                     ▼                         ▼                     │
│              [Vector Search]              [Inverted Index]           │
│              (HNSW, IVF)                  (BM25 Search)              │
│                     │                         │                     │
│                     ▼                         ▼                     │
│         Results A (Top 10)             Results B (Top 10)            │
│         [doc3, doc7, doc1, ...]        [doc1, doc5, doc3, ...]       │
│                     │                         │                     │
```

```
│                        └──────────────┬──────────────┘                        │
│                                       │                                        │
│                                       ▼                                        │
│                            [Score Fusion]                                      │
│                                       │                                        │
│                            Final: [doc3, doc1, doc5, doc7, ...]               │
│                                                                                │
└────────────────────────────────────────────────────────────────────────────────┘

[점수 결합 방법]

1. RRF (Reciprocal Rank Fusion) - 가장 많이 사용

   final_score = Σ 1/(k + rank_i)

   예: doc3이 Dense에서 1위, Sparse에서 3위
       score = 1/(60+1) + 1/(60+3) = 0.0164 + 0.0159 = 0.0323

2. Linear Combination

   final_score = α × dense_score + (1-α) × sparse_score
   (보통 α = 0.5 ~ 0.7)
```

Part 6: SPLADE - 학습된 Sparse

SPLADE는 **Dense의 의미 이해** + **Sparse의 효율성**을 결합한 방식입니다.

```
┌────────────────────────────────────────────────────────────────────────────┐
│                        SPLADE vs BM25 비교                                 │
├────────────────────────────────────────────────────────────────────────────┤
│                                                                            │
│  입력: "귀여운 강아지 사진"                                                │
│                                                                            │
│  [BM25 출력]                        [SPLADE 출력]                          │
│  {                                  {                                      │
│    "귀여운": 1.2,                     "귀여운": 0.8,                       │
│    "강아지": 2.5,                     "강아지": 2.1,                       │
│    "사진": 1.8                        "사진": 1.5,                         │
│  }                                     "반려견": 1.2,  ← 의미 확장!        │
│                                        "펫": 0.9,      ← 의미 확장!        │
│                                        "동물": 0.7,    ← 의미 확장!        │
│  3개 토큰                             "이미지": 0.6   ← 의미 확장!        │
│  (문서에 있는 단어만)               }                                      │
│                                      7개 토큰 (관련 단어도 활성화)         │
│                                                                            │
├────────────────────────────────────────────────────────────────────────────┤
│                                                                            │
│  [SPLADE 동작 원리]                                                        │
│                                                                            │
│  "귀여운 강아지 사진" ──► BERT Encoder ──► 각 토큰의 중요도 + 확장         │
│                                  │                                         │
│                                  ▼                                         │
```

```
│                    어휘 전체에 대해 활성화 점수 계산                    │
│                    (30,000개 단어 각각에 점수 부여)                     │
│                              │                                          │
│                              ▼                                          │
│                    임계값 이상만 유지 (Sparse하게 만듦)                 │
│                              │                                          │
│                              ▼                                          │
│                    {강아지:2.1, 반려견:1.2, 펫:0.9, ...}                │
│                                                                         │
│  [장점]                                                                 │
│  - Inverted Index 사용 가능 (Sparse라서)                               │
│  - Dense 수준의 의미 이해 (BERT가 확장)                                │
│  - 해석 가능 (어떤 단어가 활성화되었는지 볼 수 있음)                   │
│                                                                         │
└─────────────────────────────────────────────────────────────────────────┘
```

Part 7: 저장 공간 비교

```
1백만 문서 저장 시:

┌──────────────────┬──────────────┬──────────────┬──────────────┐
│                  │   Dense      │   Sparse     │   Hybrid     │
├──────────────────┼──────────────┼──────────────┼──────────────┤
│ 문서당 크기      │   ~6 KB      │   ~0.8 KB    │   ~7 KB      │
├──────────────────┼──────────────┼──────────────┼──────────────┤
│ 1백만 문서       │   5.7 GB     │   0.75 GB    │   6.5 GB     │
├──────────────────┼──────────────┼──────────────┼──────────────┤
│ 1천만 문서       │   57 GB      │   7.5 GB     │   65 GB      │
└──────────────────┴──────────────┴──────────────┴──────────────┘

Dense는 Sparse보다 약 7.7배 더 많은 공간 필요
```

Part 8: 실무 선택 가이드

```
┌─────────────────────────────────────────────────────────────────────────┐
│                              선택 가이드                                 │
├─────────────────────────────────────────────────────────────────────────┤
│                                                                         │
│  Q: 일반적인 문서 검색?                                                 │
│  A: Dense (OpenAI, Cohere, BGE)                                         │
│     → 의미 검색이 중요한 대부분의 경우                                  │
│                                                                         │
│  Q: 전문 용어, 고유명사가 많다?                                         │
│  A: Hybrid (Dense + BM25)                                               │
│     → 의료, 법률, 기술 문서                                             │
│     → "COVID-19", "RTX 4090" 같은 정확 매칭 필요                        │
│                                                                         │
│  Q: 비용/속도가 중요하다?                                               │
│  A: Sparse (BM25) + 필요시 Dense 보조                                   │
```

```
│     → 대용량 로그 검색                                                  │
│     → 실시간 검색이 필요한 경우                                         │
│                                                                         │
│  Q: 최신 기술로 최고 성능?                                              │
│  A: SPLADE + Dense Hybrid                                               │
│     → 검색 품질이 가장 중요한 경우                                      │
│                                                                         │
├─────────────────────────────────────────────────────────────────────────┤
│                                                                         │
│  [추천 조합]                                                            │
│                                                                         │
│  기본:     Dense (OpenAI text-embedding-3-small)                        │
│  중급:     Dense + BM25 (Elasticsearch + Vector)                        │
│  고급:     Dense + SPLADE + BM25 (Vespa, Weaviate)                      │
│                                                                         │
└─────────────────────────────────────────────────────────────────────────┘
```

정리 표

항목	Dense	Sparse	Hybrid
대표 모델	OpenAI, BERT, BGE	BM25, TF-IDF, SPLADE	Dense + Sparse
벡터 형태	`[0.02, -0.08, ...]`	`{word: score}`	둘 다
차원 수	384 ~ 4096	30K ~ 100K+	둘 다
비영값 비율	100%	0.1% ~ 1%	-
강점	의미적 유사성, 동의어	키워드 매칭, 희귀어	둘의 장점
약점	희귀어, 전문용어	동의어 처리 불가	복잡성, 비용
검색 인덱스	ANN (HNSW, IVF)	Inverted Index	둘 다
저장/문서	~6KB	~0.8KB	~7KB
해석 가능	❌	☑	부분 ☑